

COSC/MATH 4340: Numerical Methods for Differential Equations

Partial Differential Equations (PDEs)

Class: COSC/MATH 4340

Semester: Spring 2024

Instructor: Dane Taylor

Office: Eng 4087

Email: dane.taylor@uwyo.edu





Introduction

Thus far we have considered *ordinary differential equations (ODEs)* a variable $y(t)$ changes in time t or systems of ODEs in which a set of variables $y_i(t)$ change in time:

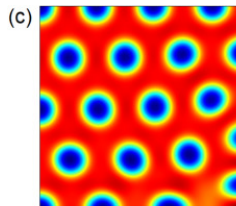
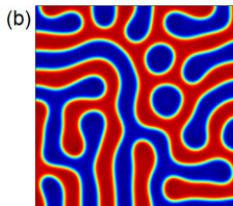
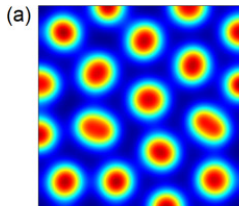
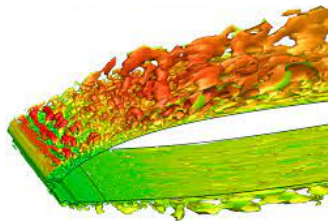
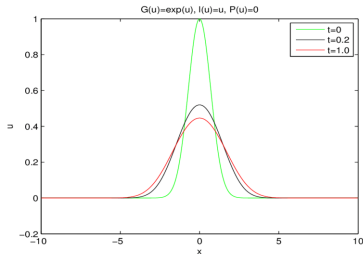
$$\frac{d}{dt}\mathbf{y} = f(t, \mathbf{y})$$

with initial condition $\mathbf{y}(t_0) = \mathbf{y}_0$ which has a solution $\mathbf{y}(t)$ over some range $t \in [t_0, T]$.

We now study *partial differential equations (PDEs)* in which a function $u(x, t)$ depends on two or more independent and continuous variables. Here, $x \in [a, b]$ often represents a spatial dimension and $t \in [0, T]$ represents time.



Introduction





Introduction

Definition

A **2nd-order partial differential equation (PDEs)** takes the form

$$Au_{xx} + Bu_{xy} + xu_{yy} + F(u_x, u_y, u, x, y) = 0,$$

where subscripts indicate partial derivatives, i.e.,

$$u_{xy} = \frac{\partial}{\partial x} \frac{\partial}{\partial y} u(x, y).$$



Types of PDEs

Given a PDE

$$Au_{xx} + Bu_{xy} + Cu_{yy} + F(u_x, u_y, u, x, y) = 0,$$

we can we can classify types of dynamics as follows:

- ▶ **Parabolic PDE** if $B^2 - 4AC = 0$: diffusion-like dynamics that dissipate with time
- ▶ **Hyperbolic PDE** if $B^2 - 4AC > 0$: wave-like dynamics that potentially oscillate
- ▶ **Elliptic PDE** if $B^2 - 4AC < 0$: curved solutions that fit boundary conditions

The Heat Equation

Definition

The heat equation is given by $u_t = Du_{xx}$, or equivalently

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2},$$

where $u(x, t)$ denotes the temperature at location x at time t and $D > 0$ is the **diffusion coefficient**

Note that the heat equation is a parabolic equation, since $B^2 - 4AC = 0$ (i.e., $B = C = 0$ and $A = D$).



The Heat Equation

The full heat equation must be specified with an initial condition and boundary conditions:

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2}$$

$$u(x, 0) = f(x), \quad x \in [a, b] \quad \text{initial temperature profile}$$

$$u(a, t) = l(t), \quad t \in [t_0, T] \quad \text{temp on the left boundary}$$

$$u(b, t) = r(t), \quad t \in [t_0, T] \quad \text{temp on the right boundary}$$



(Forward) Finite Difference Method

We discretize both the space and time domains with meshes

$$t_j = t_0 + jk, \quad j = 0, 1, \dots, N$$

$$x_i = a + ih, \quad i = 0, 1, \dots, n,$$

so that there are $N + 1$ time points with step size $k = (T - t_0)/N$ and $n + 1$ spatial points with step size $h = (b - a)/n$.

Next, we let $u(x_i, t_j)$ denote the true function at position x_i at time t_j , and we let $u_{i,j}$ denote our approximation.



(Forward) Finite Difference Method

Considering the PDE $\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2}$, we approximate both sides using finite differences:

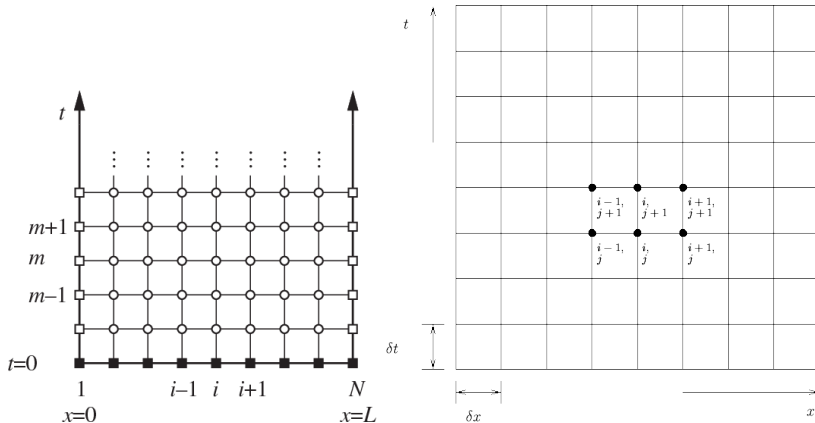
$$\begin{aligned}\frac{\partial u}{\partial t} &\approx \frac{u(x_i, t_j + k) - u(x_i, t_j)}{k} + \mathcal{O}(k) \approx \frac{u_{i,j+1} - u_{i,j}}{k} \\ \frac{\partial^2 u}{\partial x^2} &\approx \frac{u(x_i + h, t_j) - 2u(x_i, t_j) + u(x_i - h, t_j)}{h^2} + \mathcal{O}(h^2) \\ &\approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2}\end{aligned}$$

Combining these yields an “explicit” finite difference equation

$$u_{i,j+1} = \sigma u_{i+1,j} + (1 - 2\sigma)u_{i,j} + \sigma u_{i-1,j},$$

where $\sigma = Dk/h^2$. Note the approximation is 1st order in k and 2nd order in h .

(Forward) Finite Difference Method



(See notes and code for an example.)



(Forward) Finite Difference Method

Letting $\mathbf{u}_j = [u_{1,j}, u_{2,j}, \dots, u_{n-1,j}]^T$ and $\mathbf{s}_j = [u_{0,j}, 0, \dots, 0, u_{n,j}]^T$, the forward finite difference equation takes the form

$$\mathbf{u}_{j+1} = A\mathbf{u}_j + \sigma\mathbf{s}_j,$$

where

$$A = \begin{bmatrix} 1 - 2\sigma & \sigma & 0 & \dots & 0 \\ \sigma & 1 - 2\sigma & \sigma & \ddots & \vdots \\ 0 & \sigma & 1 - 2\sigma & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \sigma \\ 0 & \dots & 0 & \sigma & 1 - 2\sigma \end{bmatrix}.$$



Von Neumann Stability

Definition

The iterative equation $\mathbf{u}_{j+1} = A\mathbf{u}_j + \sigma\mathbf{s}_j$ is **Von Neumann stable** if the spectral radius is less than one: $\rho(A) < 1$.

Theorem

The Forward Finite Difference Method is stable if $\sigma < 1/2$.

Definition

A method is called **conditionally stable** if its stability depends on the step sizes h and k .

(See notes and code.)



(Backward) Finite Difference Method

Letting $\mathbf{u}_j = [u_{1,j}, u_{2,j}, \dots, u_{n-1,j}]^T$ and $\hat{\mathbf{s}}_{j+1} = [u_{0,j+1}, 0, \dots, 0, u_{n,j+1}]^T$, the backward finite difference equation takes the form

$$\hat{A}\mathbf{u}_{j+1} = \mathbf{u}_j + \sigma\hat{\mathbf{s}}_{j+1},$$

where

$$\hat{A} = \begin{bmatrix} 1 + 2\sigma & -\sigma & 0 & \dots & 0 \\ -\sigma & 1 + 2\sigma & -\sigma & \ddots & \vdots \\ 0 & -\sigma & 1 + 2\sigma & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \sigma \\ 0 & \dots & 0 & -\sigma & 1 + 2\sigma \end{bmatrix}.$$

(See notes.)



Von Neumann Stability

Theorem

The Von Neumann stability of the Backward Finite Difference Method is equivalent to the stability for the iteration $\mathbf{u}_{j+1} = \hat{A}^{-1}\mathbf{u}_j + \sigma\hat{A}^{-1}\mathbf{s}_{j+1}$, requiring $\rho(\hat{A}^{-1}) < 1$, which is guaranteed for any σ .

Definition

A method is called **unconditionally stable** if its stability does not depend on the step sizes h and k .

(See notes.)



Types of Boundary Conditions

Definition

The heat equation is said to have **Dirichlet boundary conditions** if one specifies the values at a and b : $u(a, t) = l(t)$ and $u(b, t) = r(t)$.

Definition

The heat equation is said to have **Neumann boundary conditions** if one specifies the derivatives at a and b : $u_x(a, t) = l(t)$ and $u_x(b, t) = r(t)$.

Definition

In general, setting the boundary conditions to zero is called **homogeneous boundary conditions**. For example, setting $u(a, t) = u(b, t) = 0$ or $u_x(a, t) = u_x(b, t) = 0$.



Crank-Nicolson Method

The Crank-Nicolson Method combines the forward and backward Finite Difference methods

$$\frac{\partial u}{\partial t} \approx \frac{u_{i,j+1} - u_{i,j}}{k}$$
$$\frac{\partial^2 u}{\partial x^2} \approx \frac{1}{2} \left[\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} + \frac{u_{i+1,j+1} - 2u_{i,j+1} + u_{i-1,j+1}}{h^2} \right]$$

and takes the matrix form

$$(I + \hat{A})\mathbf{u}_{j+1} = (I + A)\mathbf{u}_j + \sigma(\hat{\mathbf{s}}_{j+1} + \mathbf{s}_j),$$

(See notes.)



Crank-Nicolson Method

Theorem

The Von Neumann stability of the Crank-Nicolson Method is equivalent to the stability for the iteration

$\mathbf{u}_{j+1} = (I + \hat{A})^{-1}(I + A)\mathbf{u}_j + \sigma(I + \hat{A})^{-1}(\mathbf{s}_{j+1} + \mathbf{s}_j)$, requiring $\rho\left((I + \hat{A})^{-1}(I + A)\right) < 1$, which is guaranteed for any σ .

Theorem

The Crank-Nicolson Method is unconditionally stable

(See notes.)



Accuracy of Finite Difference Methods

- ▶ The Forward Finite Difference Method and Backward Finite Difference Method both have local truncation errors that are $\mathcal{O}(h^2) + \mathcal{O}(k)$.
- ▶ The Crank-Nicolson Method has local truncation errors that are $\mathcal{O}(h^2) + \mathcal{O}(k^2)$.

(See textbook pages 388-389 for derivation.)

The Wave Equation

Definition

The wave equation is given by $u_{tt} = c^2 u_{xx}$, or equivalently

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2},$$

where c is the **wave speed**

Describes a wave traveling in the x direction.

Note that the wave equation is a hyperbolic equation, since $B^2 - 4AC = 4c^2 > 0$.



The Wave Equation

The full wave equation must be specified with an initial condition and boundary conditions:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$$

$$u(x, 0) = f(x), \quad x \in [a, b] \quad \text{initial profile}$$

$$u_t(x, 0) = g(x), \quad x \in [a, b] \quad \text{initial derivative}$$

$$u(a, t) = l(t), \quad t \in [t_0, T] \quad \text{boundary on the left boundary}$$

$$u(b, t) = r(t), \quad t \in [t_0, T] \quad \text{boundary on the right boundary}$$

Unlike the heat equation, additional information is required to specify the initial derivative.



(Forward) Finite Difference Method

We approximately solve the wave equation $\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$ as

$$\frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{k^2} + \mathcal{O}(k^2) = c^2 \left[\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} + \mathcal{O}(h^2) \right]$$

Note that the local (or step) error is $\mathcal{O}(h^2) + \mathcal{O}(k^2)$.

Letting $\sigma = ck/h$, this system yields:

$$u_{i,j+1} = \sigma^2 u_{i+1,j} + (2 - 2\sigma^2) u_{i,j} + \sigma^2 u_{i-1,j} - u_{i,j-1}$$

(See notes.)

(Forward) Finite Difference Method

With $\mathbf{u}_j = [u_{1,j}, u_{2,j}, \dots, u_{n-1,j}]^T$ and $\mathbf{s}_j = [l(t_j), 0, \dots, 0, r(t_j)]^T$, the finite difference equation takes the form

$$\mathbf{u}_{j+1} = B\mathbf{u}_j - \mathbf{u}_{j-1} + \sigma^2 \mathbf{s}_j,$$

where

$$B = \begin{bmatrix} 2 - 2\sigma^2 & \sigma^2 & 0 & \dots & 0 \\ \sigma^2 & 2 - 2\sigma^2 & \sigma^2 & \ddots & \vdots \\ 0 & \sigma^2 & 2 - 2\sigma^2 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \sigma^2 \\ 0 & \dots & 0 & \sigma^2 & 2 - 2\sigma^2 \end{bmatrix}.$$

(See notes for derivation. Also, observe that B is identical to $I + A$ from before except now we have $\sigma \mapsto \sigma^2$.)



(Forward) Finite Difference Method

To compute the values $\mathbf{u}_{i,1}$ at the first time step t_1 , we use the derivative initial condition to obtain

$$\mathbf{u}_{j+1} = \frac{1}{2}B\mathbf{u}_j + k\mathbf{g} + \frac{1}{2}\sigma^2\mathbf{s}_j,$$

where $\mathbf{g} = [g(x_1), \dots, g(x_{n-1})]^T$. For the remaining time steps we use

$$\mathbf{u}_{j+1} = B\mathbf{u}_j - \mathbf{u}_{j-1} + \sigma^2\mathbf{s}_j.$$

(See notes.)



CFL Condition

Theorem (Courant-Friedrichs-Lewy)

The iteration

$$\mathbf{u}_{j+1} = B\mathbf{u}_j - \mathbf{u}_{j-1} + \sigma^2 \mathbf{s}_j.$$

with wave speed $c > 0$ is stable if the **CFL number** satisfies

$$\sigma = ck/h \leq 1.$$

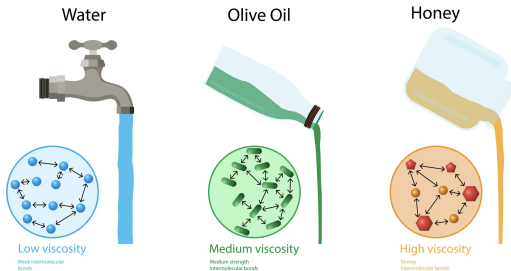
(See notes for proof.)

Nonlinear PDEs: Burgers' Equation

A slight modification of the heat equation yields a simplified fluid dynamics model, .

Definition

Burgers' equation is given by $u_t + uu_x = Du_{xx}$, where $D > 0$ introduces **viscosity**.





Burgers' Equation

The full Burgers' equation with **Dirichlet** boundary conditions is given by:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = D \frac{\partial^2 u}{\partial x^2}$$

$$u(x, 0) = f(x), \quad x \in [a, b] \quad \text{initial temperature profile}$$

$$u(a, t) = l(t), \quad t \in [t_0, T] \quad \text{temp on the left boundary}$$

$$u(b, t) = r(t), \quad t \in [t_0, T] \quad \text{temp on the right boundary}$$



Solving with Finite Difference Method

Approximate the left-hand side of Burgers' equation $u_t + uu_x = Du_{xx}$ using a backward difference for u_t and a centered difference for u_x :

$$\left[\frac{u_{i,j} - u_{i,j-1}}{k} + \mathcal{O}(k) \right] + u_{ij} \left[\frac{u_{i+1,j} - u_{i-1,j}}{2h} + \mathcal{O}(h^2) \right].$$

The right-hand side is approximated as before:

$$D \left[\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} + \mathcal{O}(h^2) \right]$$

Due to the nonlinear term uu_x , we cannot write the system of equations as a linear system, e.g., $\mathbf{A}\mathbf{u}_j = \mathbf{b}_{j-1}$



Solving with Finite Difference Method

Letting $\sigma = Dk/h^2$, the nonlinear system of equations takes the form:

$$u_{i,j} + \frac{k}{2h} u_{i,j} [u_{i+1,j} - u_{i-1,j}] - \sigma [u_{i+1,j} - 2u_{i,j} + u_{i-1,j}] - u_{i,j-1} = 0.$$

Letting $\mathbf{u}_j = [u_{0,j}, u_{1,j}, \dots, u_{n,j}]^T$, note that $\mathbf{u}_j$ must solve the nonlinear equation

$$0 = f_i(\mathbf{z}) = z_i + \frac{k}{2h} z_i [z_{i+1} - z_{i-1}] - \sigma [z_{i+1} - 2z_i + z_{i-1}] - u_{i,j-1},$$

where $u_{i,j-1}$ is known from the previous time step.

(See notes.)



Solving with Finite Difference Method

We can use a root-finding algorithm to propagate the approximate solution $\mathbf{u}_{j-1}$ at time t_{j-1} to obtain $\mathbf{u}_j$ at time t_j . (Or equivalently, from time t_j to time t_{j+1} .)

Let $F(\mathbf{z}) = [f_0(\mathbf{z}), f_1(\mathbf{z}), \dots, f_{n-1}(\mathbf{z}), f_n(\mathbf{z})]^T$ contain the functions and note that the boundary conditions require different equations:

$$0 = f_0(\mathbf{z}) = z_0 - l(t_j)$$

$$0 = f_n(\mathbf{z}) = z_n - r(t_j)$$

(See notes.)



Recall Newton's Root-finding Method

It is typically preferable to find the root using Newton's method:

Definition

Newton's method for root finding uses the iterative process

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - [J(\mathbf{x}^{(t)})]^{-1} F(\mathbf{x}^{(t)}), \quad (\text{vector } \mathbf{x})$$

where $J(\mathbf{x})$ is the **Jacobian matrix** for vector function $F(\mathbf{x}) = [f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_n(\mathbf{x})]$ in which $J_{ij}(\mathbf{x}) = \frac{\partial}{\partial x_j} f_i(\mathbf{x})$.

In practice, it's best to not invert J but instead solve the linear system:

$$J(\mathbf{x}^{(t)})\mathbf{x}^{(t+1)} = J(\mathbf{x}^{(t)})\mathbf{x}^{(t)} - F(\mathbf{x}^{(t)}).$$



Newton Method for Burger's Equation

Let $\alpha_j = -\sigma - \frac{k}{2h}z_j$, $\beta_j = 1 + 2\sigma + \frac{k}{2h}(z_{j+1} - z_{j-1})$, and $\gamma_j = -\sigma + \frac{k}{2h}z_j$. Then J is given by [page 418]

$$J = \begin{bmatrix} 1 & 0 & 0 & 0 & \dots & \dots & 0 \\ \alpha_1 & \beta_1 & \gamma_1 & & & & 0 \\ 0 & \alpha_2 & \beta_2 & \alpha_2 & & & \vdots \\ \vdots & & \ddots & \ddots & \ddots & & \vdots \\ \vdots & & & \ddots & \ddots & \ddots & 0 \\ 0 & \dots & & & \alpha_{n-1} & \beta_{n-1} & \gamma_{n-1} \\ 0 & \dots & & 0 & 0 & 0 & 1 \end{bmatrix}$$

(See notes.)



Nonlinear PDEs: Fisher's Equation

Another slight modification of the heat equation yields a model for the diffusion of population genetics

Definition

Fisher's equation is given by $u_t = Du_{xx} + g(u)$, where g is a polynomial in u .

Lemma

$x = C$ is an equilibrium to Fisher's equation wherever $g(C) = 0$ and a stable equilibrium when also $g'(C) < 0$.



Fisher's Equation

The Fisher's equation with **Neumann** boundary conditions is given by:

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2} + g(u)$$

$$u(x, 0) = f(x), \quad x \in [a, b] \quad \text{initial profile}$$

$$u_x(a, t) = l(t), \quad t \in [t_0, T] \quad \text{derivative at left boundary}$$

$$u_x(b, t) = r(t), \quad t \in [t_0, T] \quad \text{derivative at right boundary}$$



Solving with Finite Difference Method

Using backward difference for the the left-hand side, we obtain

$$\frac{u_{i,j} - u_{i,j-1}}{k} + \mathcal{O}(k) = D \left[\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} + \mathcal{O}(h^2) \right] + u_{i,j} - u_{i,j}^2$$

which simplifies to

$$\begin{aligned} u_{i,j} &= \sigma[u_{i+1,j} - 2u_{i,j} + u_{i-1,j}] + k[u_{i,j} - u_{i,j}^2] + u_{i,j-1} \\ 0 &= (1 + 2\sigma - k + ku_{i,j})u_{i,j} - \sigma u_{i+1,j} - \sigma u_{i-1,j} + u_{i,j-1}. \end{aligned}$$

To consider the left/right boundaries, we must have

$u_x(0) = u_x(1) = 1$. We use a 2nd-order approximation for the derivatives:

$$\begin{aligned} \frac{\partial}{\partial x} u(0, t_j) &= \frac{-3u_{0,j} + 4u_{1,j} - u_{2,j}}{2h} + \mathcal{O}(h^2) \\ \frac{\partial}{\partial x} u(1, t_j) &= \frac{-3u_{n,j} + 4u_{n-1,j} - u_{n-2,j}}{2h} + \mathcal{O}(h^2). \end{aligned}$$

Newton Method for Fisher's Equation

Let $\beta_j = 1 + 2\sigma - k(1 - 2z_j)$. Then J is given by [page 422]

$$J = \begin{bmatrix} -3 & 4 & -1 & 0 & \dots & \dots & 0 \\ -\sigma & \beta_1 & -\sigma & & & & 0 \\ 0 & -\sigma & \beta_2 & -\sigma & & & \vdots \\ \vdots & & \ddots & \ddots & \ddots & & \vdots \\ \vdots & & & \ddots & \ddots & \ddots & 0 \\ 0 & \dots & & & -\sigma & \beta_{n-1} & -\sigma \\ 0 & \dots & & 0 & -1 & 4 & -3 \end{bmatrix}$$



Solving Fisher's Equation with Backward Finite Differences

Example Fisher's equation.

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2} + u(1 - u)$$

$$u(x, 0) = \sin(\pi x), \quad x \in [0, 1] \quad \text{initial profile}$$

$$u_x(a, t) = 0, \quad t \in [0, 3] \quad \text{derivative at left boundary}$$

$$u_x(b, t) = 0, \quad t \in [0, 3] \quad \text{derivative at right boundary}$$

Let $D = 1$, $h = k = 0.01$.

(See code.)



Elliptic PDEs: Poisson and Laplace Equations

Definition

Let $u(x, y)$ be twice differentiable with $x \in \mathcal{X} = [x_a, x_b]$ and $y \in \mathcal{Y} = [y_a, y_b]$, and define $f(x, y)$ for $(x, y) \in \mathcal{X} \times \mathcal{Y}$. The **Poisson Equation** is given by

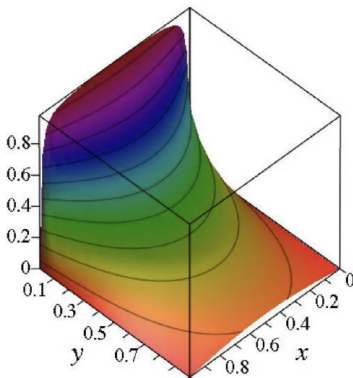
$$u_{xx} + u_{yy} = f(x, y).$$

For the choice $f(x, y) = 0$, the Poisson equation becomes the **Laplace Equation**.

These represent a third type of PDE, elliptic equations, which are time independent.

Elliptic PDEs: Laplace Equation

Example application: Solving for the steady state temperature profile when there are different boundary conditions





Laplace Equation

The 2-dimensional Laplace equation $\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0$ with Dirichlet boundary conditions has four boundary conditions

$$u(x, y_b) = g_1(x), \quad \text{temperature on bottom}$$

$$u(x, y_t) = g_2(x), \quad \text{temperature on top}$$

$$u(x, y_l) = g_3(y), \quad \text{temperature on left}$$

$$u(x, y_r) = g_4(y), \quad \text{temperature on right}$$

Note there is no initial condition since the solution $u(x, y)$ does not depend on time.

Solving with Finite Difference Method

We can approximately solve the 2-dimensional PDE by approximating it with a linear system based on finite differences:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{k^2} + \mathcal{O}(h^2, k^2),$$

where $x_i = x_0 + hi$ is a mesh in the x direction with step size h and $y_j = y_0 + kj$ is a mesh in the y direction with step size y .

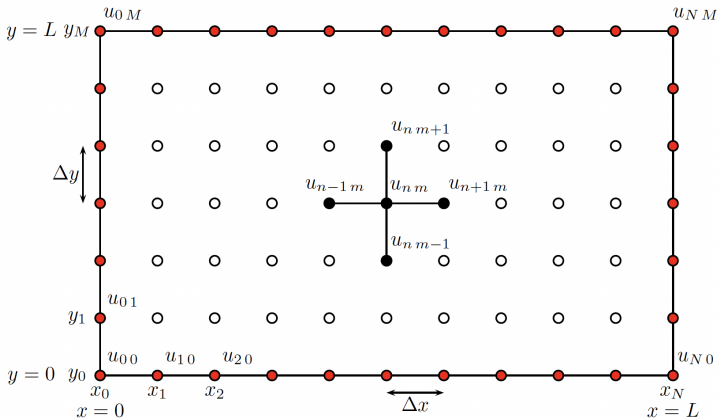
We typically choose $h = k$, yielding a system of $(n-1) \times (n-1)$ equations of the form:

$$u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j} = h^2 f(x_i, x_j)$$

where $i, j = 1, 2, \dots, n-1$.

Mesh for 2D Elliptic PDEs

2D mesh and stencil for Poisson equation.





Solving with Finite Difference Method

The linear equations

$$u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j} = h^2 f(x_i, x_j)$$

for $i, j \in \{1, 2, \dots, n-1\}$ can be rewritten in the form

$$A\hat{\mathbf{u}} = \hat{\mathbf{b}}$$

by stacking the $u_{i,j}$ values into a very long vector $\hat{\mathbf{u}} \in \mathbb{R}^{(n-1)^2}$.

Similarly, $\hat{\mathbf{b}} \in \mathbb{R}^{(n-1)^2}$ encodes the boundary conditions and $h^2 f(x_i, y_j)$ values. Matrix $A \in \mathbb{R}^{(n-1)^2 \times (n-1)^2}$ is very large and sparse.

(See notes.)



Laplace Equation Example

EXAMPLE: 2-dimensional Laplace equation $\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0$ with four Dirichlet boundary conditions

$$u(x, y_b) = g_1(x) = \sin(2\pi x)$$

$$u(x, y_t) = g_2(x) = \sin(2\pi x)$$

$$u(x, y_l) = g_3(y) = 2 \sin(2\pi y)$$

$$u(x, y_r) = g_4(y) = 2 \sin(2\pi y)$$

(See code.)